claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a787b96725eff71e5.jsonl`
- SHA-256: `1f4a46ce970b5a6b179a48f20d23463d131db0a35ba5790e1356412a83ee7a54`
- Source modified: `2026-06-10T06:12:09+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:09:28.021Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("collector") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"int(float()) rally_number truncation crashes mid-import with an uncaught IntegrityError, leaving the DB half-updated","severity":"high","area":"import-local / parsing","file":"C:/Users/avidu/Projects/sports-data-collector/src/core/parsing.py:9-19","evidence":"safe_int('7.5') == 7 (verified). A CSV containing both rally 7 and 7.5 parses to two rallies numbered 7; DatasetValidator checks overlap/duration but never duplicate rally_numbers (validator.py:29-57), so validation passes; the duplicate then violates PRIMARY KEY (video_id, rally_number) inside insert_badminton_rallies. Verified empirically: sqlite3.IntegrityError raised; the rally transaction rolls back (pre-existing rows preserved ? good), BUT in import_local the rally insert (curate.py:124-136) is OUTSIDE the parse try/except (81-86), so the user gets a raw traceback AND insert_video (curate.py:124) has already committed ? a CURATED video row with stale/zero annotations remains. The normalize_annotations.py guard fixes exactly this but is opt-in: import-local neither requires nor suggests it, so a raw vendor/annotator CSV imported directly still hits the landmine.","why_it_matters":"The two-landmine guard exists but is not wired into the import path it protects; the unguarded path fails with an inconsistent DB state (committed video metadata, missing segments) rather than a clean rejection. Note: the positioning doc's claim that the collision 'silently destroys' a rally is inaccurate for same-file imports ? it crashes loudly; the silent case is confined to a lone fractional id with no integer twin (mislabeled, not lost)."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge

severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and
a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:09:35.569Z)

```
1       """Shared parsing helpers used by the CLI importer and dataset parsers.
2
3       Previously these were duplicated as private ``_safe_int`` / ``_safe_float`` /
4       ``_parse_time_string`` in both ``src/cli/curate.py`` and the CoachAI parser.
5       """
6       from typing import Any, Optional
7
8
9       def safe_int(value: Any, default: Optional[int] = 0) -> Optional[int]:
10          """Parse an int from a possibly-empty/float/None value; return default on
failure."""
11          if value is None:
12              return default
13          s = str(value).strip()
14          if not s:
15              return default
16          try:
17              return int(float(s))
18          except (TypeError, ValueError):
19              return default
20
21
22      def safe_float_required(value: Any) -> float:
23          """Parse a float; raise ValueError if missing/empty (caller treats it as
required)."""
24          if value is None:
25              raise ValueError("missing required numeric value")
26          s = str(value).strip()
27          if not s:
28              raise ValueError("missing required numeric value")
29          return float(s)
30
31
32      def parse_time_string(time_str: Any) -> float:
33          """Convert 'HH:MM:SS' / 'MM:SS' / float-strings to float seconds; 0.0 if
blank/invalid."""
34          if not time_str:
35              return 0.0
36          s = str(time_str).strip()
37          if ":" in s:
38              parts = s.split(":")
39              try:
40                  if len(parts) == 3:
41                      return float(parts[0]) * 3600 + float(parts[1]) * 60 + float(parts[2])
42                  if len(parts) == 2:
43                      return float(parts[0]) * 60 + float(parts[1])
44              except ValueError:
45                  return 0.0
46          try:
47              return float(s)
48          except ValueError:
49              return 0.0
50
```

3. user (2026-06-10T06:09:36.248Z)

```
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tennis.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\cricket.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\base_parser.py
C:\Users\avidu\Projects\sports-data-collector\src\core\downloader.py
```

C:\Users\avidu\Projects\sports-data-collector\tests\test_downloader.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_local_import.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_schemas.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_validator.py
C:\Users\avidu\Projects\sports-data-collector\cleanup.py
C:\Users\avidu\Projects\sports-data-collector\src\cli\ingest.py
C:\Users\avidu\Projects\sports-data-collector\src\core\config.py
C:\Users\avidu\Projects\sports-data-collector\src\core\parsing.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\base.py
C:\Users\avidu\Projects\sports-data-collector\src\core\validator.py
C:\Users\avidu\Projects\sports-data-collector\src\crawlers\badminton_crawler.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\badminton\coachai.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_cli.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_config.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_parser.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_parsing.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat__init__.py
C:\Users\avidu\Projects\sports-data-collector\src\core\db.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\badminton.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\pickleball.py
C:\Users\avidu\Projects\sports-data-collector\src\core\schemas\tabletennis.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_db.py
C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_cvat_parser.py
C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py
C:\Users\avidu\Projects\sports-data-collector\src\reporting__init__.py
C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_reporting.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_reporting_faq.py
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py
C:\Users\avidu\Projects\sports-data-collector\tests\test_normalize_annotations.py

4. user (2026-06-10T06:09:43.046Z)

```
1       from typing import List, Tuple, Union
2       from src.core.config import load_config
3       from src.core.schemas.base import VideoMetadata, LicenseType
4       from src.core.schemas.badminton import BadmintonRally
5       from src.core.schemas.tennis import TennisPoint
6       from src.core.schemas.cricket import CricketDelivery
7       from src.core.schemas.pickleball import PickleballRally
8       from src.core.schemas.tabletennis import TableTennisRally
9
10      class DatasetValidator:
11          def __init__(self, config_path: str = "config/config.yaml"):
12              self.config_path = config_path
13              self.commercial_licenses = self._load_licenses_whitelist()
14
15          def _load_licenses_whitelist(self) -> List[str]:
16              # load_config returns {} on a missing/unreadable file, so the default list
17              # below is the single fallback (keep in sync with config.yaml).
18              config = load_config(self.config_path)
19              return config.get(
20                  'commercial_licenses',
21                  ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
"Proprietary"],
22              )
23
24          def check_license_commercial(self, license_type: Union[LicenseType, str]) -> bool:
25              """Determines if a license type is safe for commercial use based on the
configuration whitelist."""
26              val = license_type.value if isinstance(license_type, LicenseType) else
license_type
27              return val in self.commercial_licenses
28
```

```
29          def validate_badminton_rallies(self, rallies: List[BadmintonRally]) -> Tuple[bool,
List[str]]:
30              """
31              Validates badminton rallies:
32              - Sorted chronologically.
33              - Start time < End time (handled by Pydantic, but double checked).
34              - No temporal overlap between sequential rallies.
35              """
36              errors = []
37              if not rallies:
38                  return True, errors
39
40              # Sort by rally number to check progression
41              sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
42
43              for i in range(len(sorted_rallies)):
44                  r = sorted_rallies[i]
45                  if r.start_time >= r.end_time:
46                      errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")
47
48                  if i > 0:
49                      prev = sorted_rallies[i - 1]
50                      if r.start_time < prev.end_time:
51                          errors.append(
52                              f"Rally overlap detected between Rally {prev.rally_number} "
53                              f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
54                              f"(starts at {r.start_time}s)"
55                          )
56
57              return len(errors) == 0, errors
58
59          def validate_tennis_points(self, points: List[TennisPoint]) -> Tuple[bool,
List[str]]:
60              errors = []
61              if not points:
62                  return True, errors
63
64              sorted_pts = sorted(points, key=lambda p: p.point_number)
65              for i in range(len(sorted_pts)):
66                  pt = sorted_pts[i]
67                  if pt.start_time >= pt.end_time:
68                      errors.append(f"Point {pt.point_number}: start ({pt.start_time}) >= end
({pt.end_time})")
69
70                  if i > 0:
71                      prev = sorted_pts[i - 1]
72                      if pt.start_time < prev.end_time:
73                          errors.append(
74                              f"Point overlap detected: Point {prev.point_number} (ends
{prev.end_time}s) "
75                              f"and Point {pt.point_number} (starts {pt.start_time}s)"
76                          )
77              return len(errors) == 0, errors
78
79          def validate_cricket_deliveries(self, deliveries: List[CricketDelivery]) ->
Tuple[bool, List[str]]:
80              errors = []
81              if not deliveries:
82                  return True, errors
83
84              # Sort by over, then ball within over
85              sorted_delivs = sorted(deliveries, key=lambda d: (d.over, d.ball))
86              for i in range(len(sorted_delivs)):
87                  d = sorted_delivs[i]
```

```
                if d.start_time >= d.end_time:
                    errors.append(f"Over {d.over} Ball {d.ball}: start ({d.start_time}) >=
end ({d.end_time})")

                if i > 0:
                    prev = sorted_delivs[i - 1]
                    if d.start_time < prev.end_time:
                        errors.append(
                            f"Delivery overlap: Over {prev.over} Ball {prev.ball} (ends
{prev.end_time}s) "
                            f"and Over {d.over} Ball {d.ball} (starts {d.start_time}s)"
                        )
            return len(errors) == 0, errors

        def validate_pickleball_rallies(self, rallies: List[PickleballRally]) -> Tuple[bool,
List[str]]:
            """Validates pickleball rallies: positive durations and no temporal overlap
            between chronologically-ordered rallies (same contract as badminton)."""
            errors = []
            if not rallies:
                return True, errors

            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
            for i in range(len(sorted_rallies)):
                r = sorted_rallies[i]
                if r.start_time >= r.end_time:
                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")

                if i > 0:
                    prev = sorted_rallies[i - 1]
                    if r.start_time < prev.end_time:
                        errors.append(
                            f"Rally overlap detected between Rally {prev.rally_number} "
                            f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
                            f"(starts at {r.start_time}s)"
                        )
            return len(errors) == 0, errors

        def validate_tabletennis_rallies(self, rallies: List[TableTennisRally]) ->
Tuple[bool, List[str]]:
            """Validates table-tennis rallies: positive durations and no temporal overlap
            between chronologically-ordered rallies (same contract as badminton)."""
            errors = []
            if not rallies:
                return True, errors

            sorted_rallies = sorted(rallies, key=lambda r: r.rally_number)
            for i in range(len(sorted_rallies)):
                r = sorted_rallies[i]
                if r.start_time >= r.end_time:
                    errors.append(f"Rally {r.rally_number}: start_time ({r.start_time}) >=
end_time ({r.end_time})")

                if i > 0:
                    prev = sorted_rallies[i - 1]
                    if r.start_time < prev.end_time:
                        errors.append(
                            f"Rally overlap detected between Rally {prev.rally_number} "
                            f"(ends at {prev.end_time}s) and Rally {r.rally_number} "
                            f"(starts at {r.start_time}s)"
                        )
            return len(errors) == 0, errors

```

5. user (2026-06-10T06:09:43.420Z)

```
1       import os
2       import sys
3       import time
4       import json
5       import csv
6       import argparse
7       from typing import List, Dict, Any, Tuple
8       from tabulate import tabulate
9
10      from src.core.config import load_config
11      from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
12      from src.core.db import SportsDatabase
13      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
14      from src.core.schemas.badminton import BadmintonRally
15      from src.core.schemas.tennis import TennisPoint
16      from src.core.schemas.cricket import CricketDelivery
17      from src.core.schemas.pickleball import PickleballRally
18      from src.core.schemas.tabletennis import TableTennisRally
19      from src.core.validator import DatasetValidator
20      from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label
21
22
23      class SportsDataCLI:
24          def __init__(self, config_path: str = "config/config.yaml"):
25              self.config_path = config_path
26              self.config = self._load_config()
27
28              # Resolve DB path
29              db_path = self.config.get("database_path", "data/sports_metadata.db")
30              self.db = SportsDatabase(db_path)
31              self.validator = DatasetValidator(config_path)
32
33          def _load_config(self) -> Dict[str, Any]:
34              if not os.path.exists(self.config_path):
35                  print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
36              return load_config(self.config_path)
37
38          def status(self):
39              """Displays status summary of all videos in the database."""
40              # Query total count by status via the public DB read API.
41              rows = self.db.get_status_counts()  # list of (sport, status, cnt)
42
43              if not rows:
44                  print("\nDatabase is currently empty. Run crawlers or import local data.\n")
45                  return
46
47              data = [[sport, status, cnt] for sport, status, cnt in rows]
48              print("\n=== Dataset Ingestion Status ===")
49              print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
50              print()
51
52          def import_local(self, args):
53              """Manually registers a local video and annotation file into the database."""
54              sport_str = args.sport.lower()
55              if sport_str not in self.config.get("supported_sports", ["badminton"]):
56                  print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
57                  sys.exit(1)
58
59              # Validate video path
60              if not os.path.exists(args.video_path):
```

```
                print(f"Warning: Local video path '{args.video_path}' does not exist on disk, but continuing registration.")

            # Validate annotation file
            if not os.path.exists(args.annotations_path):
                print(f"Error: Annotations file '{args.annotations_path}' not found.")
                sys.exit(1)

            match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
            video_id = args.video_id or match_name.lower().replace(" ", "_")

            # Parse license
            try:
                license_type = LicenseType(args.license)
            except ValueError:
                print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown (non-commercial).")
                license_type = LicenseType.UNKNOWN

            is_commercial = self.validator.check_license_commercial(license_type)

            # 1. Parse Annotations
            annotations = []
            try:
                annotations = self._parse_local_annotations(sport_str, video_id, args.annotations_path)
            except Exception as e:
                print(f"Error parsing annotations: {e}")
                sys.exit(1)

            # 2. Run Validation
            is_valid = True
            errors = []
            if sport_str == "badminton":
                is_valid, errors = self.validator.validate_badminton_rallies(annotations)
            elif sport_str == "tennis":
                is_valid, errors = self.validator.validate_tennis_points(annotations)
            elif sport_str == "cricket":
                is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
            elif sport_str == "pickleball":
                is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
            elif sport_str == "tabletennis":
                is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)

            if not is_valid:
                print(f"Error: Annotations failed validation!")
                for err in errors:
                    print(f"  - {err}")
                sys.exit(1)

            # 3. Create Video Metadata
            video_meta = VideoMetadata(
                video_id=video_id,
                sport=SportType(sport_str),
                video_url=None,
                local_path=args.video_path,
                license_type=license_type,
                license_url=None,
                is_commercial=is_commercial,
                status=CurationStatus.CURATED,  # Custom local imports bypass staging by default
                match_name=match_name,
                fps=args.fps,
                resolution=args.resolution
            )
```

```
122
123             # 4. Insert into Database
124             self.db.insert_video(video_meta)
125
126             count = 0
127             if sport_str == "badminton":
128                 count = self.db.insert_badminton_rallies(annotations)
129             elif sport_str == "tennis":
130                 count = self.db.insert_tennis_points(annotations)
131             elif sport_str == "cricket":
132                 count = self.db.insert_cricket_deliveries(annotations)
133             elif sport_str == "pickleball":
134                 count = self.db.insert_pickleball_rallies(annotations)
135             elif sport_str == "tabletennis":
136                 count = self.db.insert_tabletennis_rallies(annotations)
137
138             print(f"\nSuccessfully imported local video '{match_name}':")
139             print(f"  - Video ID: {video_id}")
140             print(f"  - Sport: {sport_str}")
141             print(f"  - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
142             print(f"  - Registered {count} annotated segments.\n")
143
144             # Per-video observability report (next to the video). Never raises.
145             from src.reporting import emit_import_report
146             src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
147             report_paths = emit_import_report(
148                 video_meta_obj=video_meta, annotations=annotations, is_valid=True,
149                 errors=[], count=count, sport=sport_str, source_format=src_fmt,
150                 fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
151             )
152             if report_paths:
153                 print(f"  - Observability report: {report_paths.get('technical_md')}")
154                 print(f"  - Customer summary    : {report_paths.get('customer_md')}\n")
155
156         def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
157             """Parses simple JSON or CSV annotations files."""
158             parsed_items = []
159             ext = os.path.splitext(filepath)[1].lower()
160
161             if ext == '.json':
162                 with open(filepath, 'r', encoding='utf-8') as f:
163                     data = json.load(f)
164
165                 if not isinstance(data, list):
166                     raise ValueError("JSON file must contain a list of objects.")
167
168                 # Uses the same safe casts as the CSV path: a missing/blank required
169                 # time raises a clear ValueError (caught by the caller) instead of a
170                 # bare KeyError, and optional ints degrade to sensible defaults.
171                 for idx, obj in enumerate(data):
172                     if sport == "badminton":
173                         parsed_items.append(BadmintonRally(
174                             video_id=video_id,
175                             rally_number=_safe_int(obj.get("rally_number"), idx + 1),
176                             start_time=_safe_float(obj.get("start_time")),
177                             end_time=_safe_float(obj.get("end_time")),
178                             score_before=obj.get("score_before"),
179                             score_after=obj.get("score_after"),
180                             shots_count=_safe_int(obj.get("shots_count"), None),
181                             ending_reason=obj.get("ending_reason"),
182                             last_shot_outcome=obj.get("last_shot_outcome")
183                         ))
```

```
                    elif sport == "tennis":
                        parsed_items.append(TennisPoint(
                            video_id=video_id,
                            point_number=_safe_int(obj.get("point_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            set_score=obj.get("set_score"),
                            game_score=obj.get("game_score"),
                            server=obj.get("server"),
                            ending_type=obj.get("ending_type")
                        ))
                    elif sport == "cricket":
                        parsed_items.append(CricketDelivery(
                            video_id=video_id,
                            over=_safe_int(obj.get("over"), 0),
                            ball=_safe_int(obj.get("ball"), 0),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            batsman=obj.get("batsman"),
                            bowler=obj.get("bowler"),
                            runs=_safe_int(obj.get("runs"), None),
                            wicket=bool(obj.get("wicket", False)),
                            extras=_safe_int(obj.get("extras"), 0)
                        ))
                    elif sport == "pickleball":
                        parsed_items.append(PickleballRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tabletennis":
                        parsed_items.append(TableTennisRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
            elif ext == '.csv':
                with open(filepath, 'r', encoding='utf-8') as f:
                    reader = csv.DictReader(f)
                    # Normalize headers
                    reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []

                    for idx, row in enumerate(reader):
                        if sport == "badminton":
                            parsed_items.append(BadmintonRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
```

```
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
                        elif sport == "tennis":
                            parsed_items.append(TennisPoint(
                                video_id=video_id,
                                point_number=_safe_int(row.get("point_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                set_score=row.get("set_score"),
                                game_score=row.get("game_score"),
                                server=row.get("server"),
                                ending_type=row.get("ending_type")
                            ))
                        elif sport == "cricket":
                            parsed_items.append(CricketDelivery(
                                video_id=video_id,
                                over=_safe_int(row.get("over"), 0),
                                ball=_safe_int(row.get("ball"), 0),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                batsman=row.get("batsman"),
                                bowler=row.get("bowler"),
                                runs=_safe_int(row.get("runs"), None),
                                wicket=(row.get("wicket") or "").lower() in ["true", "1",
"yes"],
                                extras=_safe_int(row.get("extras"), 0)
                            ))
                        elif sport == "pickleball":
                            parsed_items.append(PickleballRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
                        elif sport == "tabletennis":
                            parsed_items.append(TableTennisRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
            else:
                raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")

            return parsed_items

        def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
            """Download one video at best resolution, retrying transient failures.

            A below-min-height result almost always means YouTube rate-limited the
            request and served a low fallback (not that the video lacks HD), so we
            retry with a growing backoff. Returns (DownloadResult, None) on success,
            or (None, failure_reason) once retries are exhausted. A below-min file is
```

```
            discarded each attempt so it never lands in the DB.
            """
            attempts = max(1, args.retries + 1)
            reason = "unknown error"
            for i in range(attempts):
                result = download_best_video(
                    url, dest_path,
                    max_height=max_height, max_size_mb=max_size_mb,
                    cookies_from_browser=args.cookies_from_browser,
                    cookies_file=args.cookies,
                    player_client=args.player_client,
                )
                if result.success and (result.height is None or result.height >=
args.min_height):
                    return result, None

                if not result.success:
                    reason = result.error or "unknown error"
                else:
                    reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
                              f"YouTube served a low format (rate-limited?)")
                    try:
                        os.remove(result.path)
                    except OSError:
                        pass

                if i < attempts - 1:
                    backoff = args.retry_backoff * (i + 1)
                    print(f"  attempt {i+1}/{attempts} failed: {reason}\n  retrying in
{backoff:.0f}s...")
                    time.sleep(backoff)
            return None, reason

        def fetch(self, args):
            """(Re)download videos at best available resolution, updating the DB in place.

            This is the in-place upgrade path: rows, rally annotations, and curation
            status are all preserved ? only the video file plus its `local_path`,
            `resolution`, and `fps` are refreshed. By default it skips videos that
            already have a local file at or above `--min-height`, so re-runs are cheap
            and resumable; `--force` re-fetches everything.
            """
            storage = self.config.get("video_storage", {})
            local_dir = storage.get("local_dir", "./data/videos")
            max_download_size_mb = storage.get("max_download_size_mb")
            max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
            os.makedirs(local_dir, exist_ok=True)

            commercial_only = not args.include_noncommercial
            if args.status == "all":
                statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
            else:
                statuses = [CurationStatus(args.status)]

            supported = self.config.get("supported_sports", ["badminton"])
            sport_strs = [args.sport] if args.sport else supported

            # Collect target videos.
            videos = []
            for sport_str in sport_strs:
                try:
                    sport = SportType(sport_str.lower())
                except ValueError:
```

```
                    print(f"Warning: skipping unknown sport '{sport_str}'.")
                    continue
                for status in statuses:
                    videos.extend(self.db.get_videos_by_status(status, sport))
            if args.video_id:
                videos = [v for v in videos if v.video_id == args.video_id]

            if not videos:
                print("\nNo matching videos to fetch.\n")
                return

            results = []  # (video_id, action, detail)
            for video in videos:
                if commercial_only and not video.is_commercial:
                    results.append((video.video_id, "skip", "non-commercial"))
                    continue
                if not video.video_url or "watch?v=example" in video.video_url:
                    # Either no URL, or the parser's placeholder (the ShuttleSet catalog
                    # had no URL for this match) ? nothing to download.
                    results.append((video.video_id, "skip", "no real source URL"))
                    continue

                dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")

                # Skip if we already have a good-enough local file (unless --force).
                if not args.force and os.path.exists(dest_path):
                    _, height, _ = probe_resolution_fps(dest_path)
                    if height is not None and height >= args.min_height:
                        results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
                        continue

                print(f"Fetching {video.video_id} ({video.match_name})...")
                result, fail_reason = self._fetch_one_with_retry(
                    video.video_url, dest_path, args, max_height, max_download_size_mb)
                if fail_reason is not None:
                    results.append((video.video_id, "FAIL", fail_reason))
                    if args.sleep:
                        time.sleep(args.sleep)
                    continue

                # Update only the file-derived fields; preserve everything else (UPSERT
                # mutates the row in place, leaving rallies intact).
                video.local_path = result.path
                if result.resolution_label:
                    video.resolution = result.resolution_label
                if result.fps:
                    video.fps = result.fps
                self.db.insert_video(video)

                detail = (f"{result.resolution_label or '?'}"
                          f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
                results.append((video.video_id, "fetched", detail))
                if args.sleep:
                    time.sleep(args.sleep)

            print("\n=== Fetch summary ===")
            print(tabulate([[vid, action, detail] for vid, action, detail in results],
                           headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
            fetched = sum(1 for _, a, _ in results if a == "fetched")
            failed = [r for r in results if r[1] == "FAIL"]
            print(f"\nFetched/updated : {fetched}   Already-good : {sum(1 for _,a,_ in
results if a=='have')}   "
                  f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}   Failed :
```

```
{len(failed)}")
            if failed:
                print("Failures (left unchanged in DB):")
                for vid, _, detail in failed:
                    print(f"  - {vid}: {detail}")
            print()

        def export_eval_set(self, args):
            """Export curated annotations as indexer-ready eval/training sets.

            Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
            indexer's `start,end` ground-truth format (decimal seconds), plus a single
            `<out_dir>/manifest.json` that pairs each CSV with its video file. The
            manifest is the contract the downstream eval/training driver reads: the
            indexer keys videos by *file MD5*, not by our `video_id`, so it must run
            against the exact `local_path`/`video_url` recorded here.

            By default only CURATED + commercially-safe videos are exported, mirroring
            the license gate already enforced elsewhere in the toolchain.
            """
            out_dir = args.out_dir
            commercial_only = not args.include_noncommercial
            statuses = [CurationStatus.CURATED]
            if args.include_staging:
                statuses.append(CurationStatus.STAGING)

            supported = self.config.get("supported_sports", ["badminton"])
            sport_strs = [args.sport] if args.sport else supported

            manifest = []
            skipped_noncomm = 0
            skipped_empty = 0

            for sport_str in sport_strs:
                try:
                    sport = SportType(sport_str.lower())
                except ValueError:
                    print(f"Warning: skipping unknown sport '{sport_str}'.")
                    continue

                videos = []
                for status in statuses:
                    videos.extend(self.db.get_videos_by_status(status, sport))

                for video in videos:
                    if commercial_only and not video.is_commercial:
                        skipped_noncomm += 1
                        continue

                    intervals = self.db.get_segment_intervals(video.video_id, sport)
                    if not intervals:
                        skipped_empty += 1
                        continue

                    sport_dir = os.path.join(out_dir, sport.value)
                    os.makedirs(sport_dir, exist_ok=True)
                    csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
                    with open(csv_path, "w", newline="", encoding="utf-8") as f:
                        writer = csv.writer(f)
                        writer.writerow(["start", "end"])
                        for start, end in intervals:
                            writer.writerow([start, end])

                    manifest.append({
                        "video_id": video.video_id,
```

```
                        "sport": sport.value,
                        "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
                        "local_path": video.local_path,
                        "video_url": video.video_url,
                        "match_name": video.match_name,
                        "fps": video.fps,
                        "resolution": video.resolution,
                        "license_type": video.license_type.value,
                        "is_commercial": video.is_commercial,
                        "status": video.status.value,
                        "num_segments": len(intervals),
                    })

            os.makedirs(out_dir, exist_ok=True)
            manifest_doc = {
                "commercial_only": commercial_only,
                "statuses": [s.value for s in statuses],
                "total_videos": len(manifest),
                "total_segments": sum(m["num_segments"] for m in manifest),
                "eval_sets": manifest,
            }
            manifest_path = os.path.join(out_dir, "manifest.json")
            with open(manifest_path, "w", encoding="utf-8") as f:
                json.dump(manifest_doc, f, indent=2)

            print(f"\n=== Exported eval/training set to '{out_dir}' ===")
            if manifest:
                summary = [[m["sport"], m["video_id"], m["num_segments"],
                            "YES" if m["is_commercial"] else "NO",
                            os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
                           for m in manifest]
                print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
            print(f"\nVideos exported : {len(manifest)}")
            print(f"Total segments  : {manifest_doc['total_segments']}")
            if skipped_noncomm:
                print(f"Skipped (non-commercial) : {skipped_noncomm}  (pass --include-
noncommercial to include)")
            if skipped_empty:
                print(f"Skipped (no annotations) : {skipped_empty}")
            print(f"Manifest        : {manifest_path}\n")

        def convert_cvat(self, args):
            """Convert a vendor CVAT rally export into our canonical per-rally CSV.

            Absorbs the CVAT bounding-box delivery format (rally fields carried as box
            attributes) on the ingestion side: each rally's time is recomputed from
            its frame range at the video's true fps, and content defects (overlaps,
            zero-duration, off-vocabulary endings, coarse sampling) are written to an
            issues report rather than silently "fixed". The emitted CSV imports via
            `import-local`; any blocker issue must be resolved first, since the
            importer's validator rejects overlapping / zero-duration rallies.
            """
            from src.parsers.cvat.rally_cvat import (
                convert, write_canonical_csv, render_issue_report,
            )
            try:
                result = convert(args.input, fps=args.fps,
                                 video_path=args.video_path, step=args.step)
            except (FileNotFoundError, ValueError) as e:
                print(f"Error: {e}")
                sys.exit(1)

            write_canonical_csv(result.records, args.out)
```

```
560
561             # Observability report next to the canonical CSV. Never raises.
562             from src.reporting import emit_delivery_report
563             src_name = os.path.splitext(os.path.basename(args.input))[0]
564             dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
565             emit_delivery_report(
566                 out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
567                 records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
568                 video_duration_s=dur, source_format="cvat", step=result.step,
569             )
570
571             report = render_issue_report(result, os.path.basename(args.input))
572             if args.issues:
573                 with open(args.issues, "w", encoding="utf-8") as f:
574                     f.write(report)
575             print(report)
576             print(f"Canonical CSV written: {args.out}  ({len(result.records)} rallies)")
577             if result.n_blockers:
578                 print(f"\n  {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
579                       f"this file until they are resolved (by the vendor or manual
review).")
580             else:
581                 print("\n  No blockers. Next:")
582                 print(f"    python -m src.cli.curate import-local --sport <sport> "
583                       f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
584
585         def validate_delivery(self, args):
586             """Run the full validation suite on a rally delivery and emit a report.
587
588             Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
589             CSV (--csv). Writes, into --out-dir:
590               <id>_report.md    human report (scoreboard + issues + rallies to review)
591               <id>_report.json  machine summary (for CI / a bulk-ingest loop)
592               <id>_review.csv   the annotatable manual-verification sheet
593             Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
594             """
595             import json
596             from src.parsers.cvat.rally_cvat import (
597                 convert, load_canonical_csv, detect_issues,
598                 build_summary, render_validation_md, write_review_csv,
599             )
600             vid = args.video_id or "delivery"
601             fps = None
602             step = None
603             stop_frame = None
604             if args.input:
605                 try:
606                     result = convert(args.input, fps=args.fps, video_path=args.video_path)
607                 except (FileNotFoundError, ValueError) as e:
608                     print(f"Error: {e}")
609                     sys.exit(1)
610                 records, issues, fps = result.records, result.issues, result.fps
611                 step = result.step
612                 stop_frame = result.meta.get("stop_frame")
613             elif args.csv:
614                 try:
615                     records = load_canonical_csv(args.csv)
616                 except FileNotFoundError as e:
617                     print(f"Error: {e}")
618                     sys.exit(1)
619                 if not records:
```

```
                    print(f"Error: no usable rally rows in {args.csv}")
                    sys.exit(1)
                issues = detect_issues(records)
            else:
                print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
                sys.exit(1)

            os.makedirs(args.out_dir, exist_ok=True)
            summary = build_summary(records, issues, source=vid, fps=fps)
            md = render_validation_md(records, issues, source=vid)
            json_path = os.path.join(args.out_dir, f"{vid}_report.json")
            md_path = os.path.join(args.out_dir, f"{vid}_report.md")
            review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
            with open(json_path, "w", encoding="utf-8") as f:
                json.dump(summary, f, indent=2)
            with open(md_path, "w", encoding="utf-8") as f:
                f.write(md)
            write_review_csv(records, issues, review_path)

            # Observability report (envelope JSON + technical md + customer summary),
            # colocated with the existing artifacts. Never raises.
            from src.reporting import emit_delivery_report
            duration_s = (stop_frame / fps) if (stop_frame and fps) else None
            report_paths = emit_delivery_report(
                out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
                video_path=args.video_path, video_duration_s=duration_s,
                source_format=("cvat" if args.input else "csv"), step=step,
            )

            print(md)
            print(f"\nWrote:\n  {md_path}\n  {json_path}\n  {review_path}")
            if report_paths:
                print(f"  {report_paths.get('technical_md')}\n
{report_paths.get('customer_md')}\n  {report_paths.get('json')}")
            if summary["n_blockers"]:
                print(f"\n  GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
                sys.exit(2)
            print("\n  GATE: PASS ? review the flagged rallies, then `import-local`.")

        def curate(self):
            """Interactive loop to review and curate staging video annotations."""
            staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
            if not staging_videos:
                print("\nNo staging video annotations to review.\n")
                return

            print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
            table_data = []
            for idx, video in enumerate(staging_videos):
                table_data.append([
                    idx + 1,
                    video.video_id,
                    video.match_name,
                    video.sport.value,
                    video.license_type.value,
                    "YES" if video.is_commercial else "NO"
                ])

            print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
            print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))

            choice = input("> ").strip()
```

```
            if choice.lower() == 'q':
                return

            try:
                idx = int(choice) - 1
                if idx < 0 or idx >= len(staging_videos):
                    print("Invalid choice.")
                    return
                self._curate_video_interactive(staging_videos[idx])
            except ValueError:
                print("Invalid input.")

        def _curate_video_interactive(self, video: VideoMetadata):
            """Displays details of a specific staging video and prompts for curation
action."""

            # Load and count events
            rallies = []
            pts = []
            delivs = []
            if video.sport == SportType.BADMINTON:
                rallies = self.db.get_badminton_rallies(video.video_id)
            elif video.sport == SportType.PICKLEBALL:
                rallies = self.db.get_pickleball_rallies(video.video_id)
            elif video.sport == SportType.TABLE_TENNIS:
                rallies = self.db.get_tabletennis_rallies(video.video_id)
            elif video.sport == SportType.TENNIS:
                pts = self.db.get_tennis_points(video.video_id)
            elif video.sport == SportType.CRICKET:
                delivs = self.db.get_cricket_deliveries(video.video_id)

            while True:
                print("\n================ REVIEWING VIDEO ================")
                print(f"Match Name   : {video.match_name}")
                print(f"Video ID     : {video.video_id}")
                print(f"Sport        : {video.sport.value}")
                print(f"Video URL    : {video.video_url or 'N/A'}")
                print(f"Local Path   : {video.local_path or 'N/A'}")
                print(f"License      : {video.license_type.value} ({video.license_url or 'no
URL'})")
                print(f"Commercial OK: {video.is_commercial}")

                if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
                    print(f"Total Rallies: {len(rallies)}")
                    if rallies:
                        print("Sample rallies:")
                        sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
                        print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
                elif video.sport == SportType.TENNIS:
                    print(f"Total Points : {len(pts)}")
                    if pts:
                        print("Sample points:")
                        sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
                        print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
                elif video.sport == SportType.CRICKET:
                    print(f"Total Balls  : {len(delivs)}")
                    if delivs:
                        print("Sample deliveries:")
                        sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
```

```
                        print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))

                print("\nChoose Curation Action:")
                print("  [a] Approve (promote to CURATED)")
                print("  [r] Reject (delete from database)")
                print("  [p] Play a rally/point segment in browser")
                print("  [k] Keep in Staging (do nothing and go back)")

                action = input("> ").strip().lower()
                if action == 'a':
                    self.db.update_status(video.video_id, CurationStatus.CURATED)
                    print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
                    break
                elif action == 'r':
                    self.db.delete_video(video.video_id)
                    print(f"Successfully rejected and deleted '{video.match_name}'.\n")
                    break
                elif action == 'k':
                    print("Curation state unchanged.\n")
                    break
                elif action == 'p':
                    self._play_segment_interactive(video, rallies, pts, delivs)
                else:
                    print("Invalid action selected.")

        def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
            import webbrowser

            # Get start/end based on user input
            start_time = None
            end_time = None
            label = "segment"

            if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
                if not rallies:
                    print("No rallies to play.")
                    return
                idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
                try:
                    rally_num = int(idx_str)
                    rally = next((r for r in rallies if r.rally_number == rally_num), None)
                    if rally:
                        start_time = rally.start_time
                        end_time = rally.end_time
                        label = f"Rally {rally_num}"
                except ValueError:
                    pass
            elif video.sport == SportType.TENNIS:
                if not pts:
                    print("No points to play.")
                    return
                idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
                try:
                    pt_num = int(idx_str)
                    pt = next((p for p in pts if p.point_number == pt_num), None)
                    if pt:
                        start_time = pt.start_time
                        end_time = pt.end_time
                        label = f"Point {pt_num}"
                except ValueError:
                    pass
            elif video.sport == SportType.CRICKET:
```

```
                if not delivs:
                    print("No deliveries to play.")
                    return
                over_str = input("Enter Over number: ").strip()
                ball_str = input("Enter Ball number: ").strip()
                try:
                    o_num = int(over_str)
                    b_num = int(ball_str)
                    d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
                    if d:
                        start_time = d.start_time
                        end_time = d.end_time
                        label = f"Over {o_num} Ball {b_num}"
                except ValueError:
                    pass

            if start_time is None or end_time is None:
                print("Segment not found or invalid input.")
                return

            print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")

            # Check video location
            play_url = None
            if video.local_path and os.path.exists(video.local_path):
                abs_path = os.path.abspath(video.local_path)
                # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
                play_url = f"file:///{abs_path.replace(os.sep,
'/')}#t={start_time},{end_time}"
            elif video.video_url:
                # YouTube URL: seek to start time
                if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
                    t_sec = int(start_time)
                    sep = "&" if "?" in video.video_url else "?"
                    play_url = f"{video.video_url}{sep}t={t_sec}s"
                else:
                    play_url = video.video_url

            if play_url:
                print(f"Opening browser: {play_url}")
                webbrowser.open(play_url)
            else:
                print("Error: No local video file or remote URL available to play.")


    def main():
        parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
        subparsers = parser.add_subparsers(dest="command")

        # Status command
        subparsers.add_parser("status", help="Display summary status of sports metadata
database")

        # Curate command
        subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")

        # Import-local command
        import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
        import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
        import_parser.add_argument("--video-path", required=True, help="Path to local video
```

```
file")
858         import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
859         import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
860         import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
861         import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
862         import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
863         import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
864
865         # Fetch command ? (re)download videos at best resolution, update DB in place
866         fetch_parser = subparsers.add_parser(
867             "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
868         fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
869         fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
870         fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
871                                   help="Which curation statuses to fetch (default: all)")
872         fetch_parser.add_argument("--min-height", type=int, default=720,
873                                   help="Skip videos already at/above this height unless
--force (default: 720)")
874         fetch_parser.add_argument("--max-height", type=int, default=None,
875                                   help="Cap download resolution (e.g. 1080); default: best
available")
876         fetch_parser.add_argument("--force", action="store_true",
877                                   help="Re-download even if a good-enough local file already
exists")
878         fetch_parser.add_argument("--include-noncommercial", action="store_true",
879                                   help="Include videos whose license is not commercially
safe (default: commercial only)")
880         fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
881                                   help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
882                                        "A Premium login gives reliable best-resolution
downloads. "
883                                        "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
884         fetch_parser.add_argument("--cookies", metavar="FILE",
885                                   help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
886         fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
887                                   help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
888                                        "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
889         fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
890                                   help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
891         fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
892                                   help="Retries per video when a download fails or returns a
low format (default: 2)")
893         fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
894                                   help="Base backoff between retries; grows linearly per
attempt (default: 20)")
895
896         # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
897         cvat_parser = subparsers.add_parser(
```

```
            "convert-cvat",
            help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
        cvat_parser.add_argument("--input", required=True,
                                 help="Path to the CVAT XML export (image- or track-mode)")
        cvat_parser.add_argument("--out", required=True,
                                 help="Path to write the canonical per-rally CSV (import-
local input)")
        cvat_parser.add_argument("--video-path",
                                 help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
        cvat_parser.add_argument("--fps", type=float,
                                 help="Override fps instead of probing the video")
        cvat_parser.add_argument("--step", type=int,
                                 help="Frame sampling step (default: read from CVAT meta,
else inferred)")
        cvat_parser.add_argument("--issues",
                                 help="Path to also write the human-readable issues report")

        # Validate-delivery command ? run the full check suite + emit a structured report
        vd_parser = subparsers.add_parser(
            "validate-delivery",
            help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
        vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
        vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
        vd_parser.add_argument("--video-path",
                               help="Source video; fps via ffprobe (when using --input)")
        vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
        vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
        vd_parser.add_argument("--out-dir", default="data/validation",
                               help="Where to write the report + review sheet")

        # Export command
        export_parser = subparsers.add_parser(
            "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
        export_parser.add_argument("--out-dir", default="data/eval_export",
                                   help="Output directory for per-video CSVs and
manifest.json")
        export_parser.add_argument("--sport",
                                   help="Restrict to a single sport (default: all
supported_sports)")
        export_parser.add_argument("--include-staging", action="store_true",
                                   help="Also export videos still in staging (default:
curated only)")
        export_parser.add_argument("--include-noncommercial", action="store_true",
                                   help="Include videos whose license is not commercially
safe (default: commercial only)")

        args = parser.parse_args()
        cli = SportsDataCLI()

        if args.command == "status":
            cli.status()
        elif args.command == "curate":
            cli.curate()
        elif args.command == "import-local":
            cli.import_local(args)
        elif args.command == "convert-cvat":
            cli.convert_cvat(args)
```

```
949         elif args.command == "validate-delivery":
950             cli.validate_delivery(args)
951         elif args.command == "export":
952             cli.export_eval_set(args)
953         elif args.command == "fetch":
954             cli.fetch(args)
955         else:
956             # Default: print help
957             parser.print_help()
958 
959     if __name__ == "__main__":
960         main()
961 
```

6. assistant (2026-06-10T06:09:51.183Z)

Now let me check the DB schema and insert behavior, plus the normalize_annotations guard.

7. user (2026-06-10T06:09:52.146Z)

```
1       import sqlite3
2       import os
3       from contextlib import contextmanager
4       from typing import List, Dict, Any, Optional
5       from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
6       from src.core.schemas.badminton import BadmintonRally
7       from src.core.schemas.tennis import TennisPoint
8       from src.core.schemas.cricket import CricketDelivery
9       from src.core.schemas.pickleball import PickleballRally
10      from src.core.schemas.tabletennis import TableTennisRally
11
12      # Maps each sport to its annotation table and the column(s) that order segments
13      # chronologically. All child tables expose (start_time, end_time), so a
14      # single generic reader can serve every sport ? adding a new sport is one entry
15      # here plus its CREATE TABLE, with no new query code.
16      SEGMENT_TABLES = {
17          SportType.BADMINTON: ("badminton_rallies", "rally_number"),
18          SportType.TENNIS: ("tennis_points", "point_number"),
19          SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
20          SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
21          SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
22      }
23
24
25      class SportsDatabase:
26          def __init__(self, db_path: str):
27              self.db_path = db_path
28              # Ensure target directory exists
29              os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
30              self._init_db()
31
32          @contextmanager
33          def _get_connection(self):
34              """Yield a SQLite connection and always close it.
35
36              Used as ``with self._get_connection() as conn:``. A plain
37              ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
38              transaction but does NOT close the connection, which leaks handles and
39              raises ResourceWarnings. This wrapper guarantees the connection is closed
40              on exit; writers still commit explicitly inside the block.
41              """
42              conn = sqlite3.connect(self.db_path)
43              conn.row_factory = sqlite3.Row
44              # SQLite disables foreign keys per-connection by default; enable so the
45              # ON DELETE CASCADE declarations on the child tables actually fire.
```

```
            conn.execute("PRAGMA foreign_keys = ON")
            try:
                yield conn
            finally:
                conn.close()

        def _init_db(self):
            with self._get_connection() as conn:
                cursor = conn.cursor()

                # 1. Create common videos table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    video_id TEXT PRIMARY KEY,
                    sport TEXT NOT NULL,
                    video_url TEXT,
                    local_path TEXT,
                    license_type TEXT NOT NULL,
                    license_url TEXT,
                    is_commercial INTEGER NOT NULL DEFAULT 0,
                    status TEXT NOT NULL DEFAULT 'staging',
                    match_name TEXT NOT NULL,
                    fps REAL,
                    resolution TEXT
                )
                """)

                # 2. Create badminton_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS badminton_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 3. Create tennis_points table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS tennis_points (
                    video_id TEXT NOT NULL,
                    point_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    set_score TEXT,
                    game_score TEXT,
                    server TEXT,
                    ending_type TEXT,
                    PRIMARY KEY (video_id, point_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 4. Create cricket_deliveries table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS cricket_deliveries (
                    video_id TEXT NOT NULL,
                    over_num INTEGER NOT NULL,
```

```
                    ball_num INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    batsman TEXT,
                    bowler TEXT,
                    runs INTEGER,
                    wicket INTEGER DEFAULT 0,
                    extras INTEGER DEFAULT 0,
                    PRIMARY KEY (video_id, over_num, ball_num),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 5. Create pickleball_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS pickleball_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 6. Create tabletennis_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS tabletennis_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # Migration: add last_shot_outcome to rally tables that predate the
                # column. CREATE TABLE IF NOT EXISTS won't alter an existing table, so
                # do it explicitly and idempotently (skip if the column is already there).
                for tbl in ("badminton_rallies", "pickleball_rallies",
"tabletennis_rallies"):
                    existing = [r[1] for r in cursor.execute(f"PRAGMA
table_info({tbl})").fetchall()]
                    if "last_shot_outcome" not in existing:
                        cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
TEXT")

                conn.commit()

        def insert_video(self, video: VideoMetadata) -> bool:
            """Insert a video entry, or update it in place if it already exists.

            Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
            existing row, which (with foreign keys enabled) would cascade-delete the
```

```
            video's child annotations. UPSERT mutates the row in place and leaves
            children untouched.
            """
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                INSERT INTO videos (
                    video_id, sport, video_url, local_path, license_type,
                    license_url, is_commercial, status, match_name, fps, resolution
                ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                ON CONFLICT(video_id) DO UPDATE SET
                    sport=excluded.sport,
                    video_url=excluded.video_url,
                    local_path=excluded.local_path,
                    license_type=excluded.license_type,
                    license_url=excluded.license_url,
                    is_commercial=excluded.is_commercial,
                    status=excluded.status,
                    match_name=excluded.match_name,
                    fps=excluded.fps,
                    resolution=excluded.resolution
                """, (
                    video.video_id,
                    video.sport.value,
                    video.video_url,
                    video.local_path,
                    video.license_type.value,
                    video.license_url,
                    1 if video.is_commercial else 0,
                    video.status.value,
                    video.match_name,
                    video.fps,
                    video.resolution
                ))
                conn.commit()
                return True

        def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                # First clear existing rallies for this video_id to avoid key conflicts on
updates
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO badminton_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
```

```
                    ))
                conn.commit()
                return len(rallies)

        def insert_tennis_points(self, points: List[TennisPoint]) -> int:
            if not points:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = points[0].video_id
                cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))

                for pt in points:
                    cursor.execute("""
                    INSERT INTO tennis_points (
                        video_id, point_number, start_time, end_time,
                        set_score, game_score, server, ending_type
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        pt.video_id,
                        pt.point_number,
                        pt.start_time,
                        pt.end_time,
                        pt.set_score,
                        pt.game_score,
                        pt.server,
                        pt.ending_type
                    ))
                conn.commit()
                return len(points)

        def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
            if not deliveries:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = deliveries[0].video_id
                cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))

                for deliv in deliveries:
                    cursor.execute("""
                    INSERT INTO cricket_deliveries (
                        video_id, over_num, ball_num, start_time, end_time,
                        batsman, bowler, runs, wicket, extras
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        deliv.video_id,
                        deliv.over,
                        deliv.ball,
                        deliv.start_time,
                        deliv.end_time,
                        deliv.batsman,
                        deliv.bowler,
                        deliv.runs,
                        1 if deliv.wicket else 0,
                        deliv.extras
                    ))
                conn.commit()
                return len(deliveries)

        def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
```

```
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO pickleball_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO tabletennis_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def get_video(self, video_id: str) -> Optional[VideoMetadata]:
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
                row = cursor.fetchone()
                if row:
                    return VideoMetadata(
                        video_id=row['video_id'],
                        sport=SportType(row['sport']),
```

```
                        video_url=row['video_url'],
                        local_path=row['local_path'],
                        license_type=LicenseType(row['license_type']),
                        license_url=row['license_url'],
                        is_commercial=bool(row['is_commercial']),
                        status=CurationStatus(row['status']),
                        match_name=row['match_name'],
                        fps=row['fps'],
                        resolution=row['resolution']
                    )
            return None

        def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
            query = "SELECT * FROM videos WHERE status = ?"
            params = [status.value]
            if sport:
                query += " AND sport = ?"
                params.append(sport.value)

            videos = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(query, params)
                for row in cursor.fetchall():
                    videos.append(VideoMetadata(
                        video_id=row['video_id'],
                        sport=SportType(row['sport']),
                        video_url=row['video_url'],
                        local_path=row['local_path'],
                        license_type=LicenseType(row['license_type']),
                        license_url=row['license_url'],
                        is_commercial=bool(row['is_commercial']),
                        status=CurationStatus(row['status']),
                        match_name=row['match_name'],
                        fps=row['fps'],
                        resolution=row['resolution']
                    ))
            return videos

        def get_status_counts(self) -> List[tuple]:
            """Return (sport, status, count) rows for the status summary.

            Public read API so callers (e.g. the CLI `status` command) don't have to
            reach into the private `_get_connection` context manager.
            """
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
                )
                return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]

        def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM badminton_rallies
                WHERE video_id = ?
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(BadmintonRally(
```

```
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
            points = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM tennis_points
                WHERE video_id = ?
                ORDER BY point_number
                """, (video_id,))
                for row in cursor.fetchall():
                    points.append(TennisPoint(
                        video_id=row['video_id'],
                        point_number=row['point_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        set_score=row['set_score'],
                        game_score=row['game_score'],
                        server=row['server'],
                        ending_type=row['ending_type']
                    ))
            return points

        def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
            deliveries = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM cricket_deliveries
                WHERE video_id = ?
                ORDER BY over_num, ball_num
                """, (video_id,))
                for row in cursor.fetchall():
                    deliveries.append(CricketDelivery(
                        video_id=row['video_id'],
                        over=row['over_num'],
                        ball=row['ball_num'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        batsman=row['batsman'],
                        bowler=row['bowler'],
                        runs=row['runs'],
                        wicket=bool(row['wicket']),
                        extras=row['extras']
                    ))
            return deliveries

        def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM pickleball_rallies
                WHERE video_id = ?
```

```
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(PickleballRally(
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM tabletennis_rallies
                WHERE video_id = ?
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(TableTennisRally(
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
            """Return chronologically-ordered (start_time, end_time) tuples for a video,
            regardless of sport. This is the sport-agnostic view used to export
            indexer-ready evaluation/training labels.

            Sports that have no annotation table yet (e.g. tabletennis, pickleball)
            yield an empty list rather than raising.
            """
            mapping = SEGMENT_TABLES.get(sport)
            if mapping is None:
                return []
            table, order_by = mapping  # both come from the trusted map above, never user
input
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    f"SELECT start_time, end_time FROM {table} "
                    f"WHERE video_id = ? ORDER BY {order_by}",
                    (video_id,),
                )
                return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]

        def update_status(self, video_id: str, status: CurationStatus) -> bool:
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
```

```
(status.value, video_id))
                conn.commit()
                return cursor.rowcount > 0

        def delete_video(self, video_id: str) -> bool:
            """Deletes a video and cascading child entries."""
            with self._get_connection() as conn:
                cursor = conn.cursor()
                # Execute cascade deletes manually just in case SQLite foreign key support
is not active
                cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
                cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
                cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
                cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))
                cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
                cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
                conn.commit()
                return cursor.rowcount > 0

```

8. user (2026-06-10T06:09:52.435Z)

```
      """Normalize a VLC rally-annotator CSV into a clean lean CSV that ``import-local``
      ingests safely.

      Two verified landmines in PARTIAL human annotation data (a video tagged only up to
      some point, with mid-session corrections):

        1. An unusable time aborts the WHOLE import (one bad row kills the entire video,
           via the try/except around _parse_local_annotations in curate.py import-local):
           a blank/missing time makes ``safe_float_required`` raise; a reversed
           ``end <= start`` or a negative ``start`` makes the ``BadmintonRally`` pydantic
           validator raise. -> we DROP such rows, mirroring the downstream indexer loader
           which already skips ``not end > start``.
        2. A fractional or duplicate ``rally_number`` (e.g. ``7.5`` inserted for a rally
           the annotator missed) truncates via ``int(float())`` and COLLIDES on the
           ``(video_id, rally_number)`` primary key; the DELETE-then-INSERT upsert then
           silently destroys one rally. -> we RENUMBER 1..N by start_time, recording the
           original id for traceability.

      It also accepts mm:ss / HH:MM:SS times (converted to decimal seconds) and folds a
      pipe-delimited ``ending_reason`` ('forced_error|other') to its primary token, so the
      output is always the canonical lean schema::

          rally_number,start_time,end_time,ending_reason,sport[,shots_count]

      plus traceability columns (``original_rally_number``, ``notes``) that import-local
      ignores. Standalone on purpose: it does NOT touch the DB schema or the
      rally_number-ordered export. Run it, then feed the output to ``curate import-local``.

          python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
      """
      from __future__ import annotations

      import argparse
      import csv
      import os
      from dataclasses import dataclass, field
      from typing import Any, Dict, List, Optional, Tuple

      # Canonical lean output columns + traceability (import-local reads the first four/
```

```
    # five; original_rally_number + notes are ignored by the importer but kept for humans).
    OUT_FIELDS = ["rally_number", "start_time", "end_time", "ending_reason", "sport",
                  "shots_count", "original_rally_number", "notes"]


    @dataclass
    class NormalizeReport:
        rows_in: int = 0
        rows_out: int = 0
        dropped: List[Dict[str, Any]] = field(default_factory=list)  # {orig_id, reason,
start, end}
        renumbered: bool = False
        fractional_ids: List[str] = field(default_factory=list)
        duplicate_ids: List[str] = field(default_factory=list)
        mmss_converted: int = 0

        def summary(self) -> str:
            lines = [f"rows in: {self.rows_in}  ->  rows out: {self.rows_out}"]
            if self.dropped:
                lines.append(f"dropped {len(self.dropped)} bad row(s):")
                for d in self.dropped:
                    lines.append(f"  - orig id {d['orig_id']!r}: {d['reason']} "
                                 f"(start={d['start']!r}, end={d['end']!r})")
            if self.renumbered:
                extra = []
                if self.fractional_ids:
                    extra.append(f"fractional ids {self.fractional_ids}")
                if self.duplicate_ids:
                    extra.append(f"duplicate ids {self.duplicate_ids}")
                lines.append("renumbered rallies 1..N by start_time"
                             + (f" ({'; '.join(extra)})" if extra else ""))
            if self.mmss_converted:
                lines.append(f"converted {self.mmss_converted} mm:ss timestamp(s) to
seconds")
            if not self.dropped and not self.renumbered and not self.mmss_converted:
                lines.append("clean input ? no changes beyond column normalization")
            return "\n".join(lines)


    def _to_seconds(raw: Any) -> Optional[Tuple[float, bool]]:
        """(seconds, was_mmss) or None if blank/unparseable. Decimal seconds or mm:ss /
HH:MM:SS."""
        if raw is None:
            return None
        s = str(raw).strip()
        if not s:
            return None
        try:
            if ":" in s:
                parts = [float(p) for p in s.split(":")]
                if len(parts) == 3:
                    return parts[0] * 3600 + parts[1] * 60 + parts[2], True
                if len(parts) == 2:
                    return parts[0] * 60 + parts[1], True
                return None
            return float(s), False
        except ValueError:
            return None


    def _primary_reason(raw: Any) -> Tuple[str, str]:
        """Pipe-delimited 'forced_error|other' -> ('forced_error', 'other'). Blank -> ('',
'')."""
        if raw is None:
            return "", ""
```

```
        s = str(raw).strip()
        if not s:
            return "", ""
        parts = [p.strip() for p in s.split("|") if p.strip()]
        if not parts:
            return "", ""
        return parts[0], "; ".join(parts[1:])


    def _get(row: Dict[str, Any], *keys: str) -> Any:
        """First non-empty value among the given (already-lowercased) keys."""
        for k in keys:
            v = row.get(k)
            if v is not None and str(v).strip() != "":
                return v
        return None


    def normalize_rows(rows: List[Dict[str, Any]]) -> Tuple[List[Dict[str, Any]],
NormalizeReport]:
        """Pure core: lowercase-keyed annotator rows -> clean lean rows + report."""
        rep = NormalizeReport(rows_in=len(rows))
        kept: List[Dict[str, Any]] = []

        for row in rows:
            orig_id = _get(row, "rally_number", "rally")
            s_parsed = _to_seconds(_get(row, "start_time", "start_mmss", "start"))
            e_parsed = _to_seconds(_get(row, "end_time", "end_mmss", "end"))
            start = s_parsed[0] if s_parsed else None
            end = e_parsed[0] if e_parsed else None

            if start is None:
                rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
start_time",
                                    "start": _get(row, "start_time", "start_mmss"), "end":
_get(row, "end_time", "end_mmss")})
                continue
            if start < 0:
                # BadmintonRally's pydantic validator rejects start<0 -> would abort the
                # whole import; drop it here like any other unusable row.
                rep.dropped.append({"orig_id": orig_id, "reason": "negative start_time",
                                    "start": start, "end": end})
                continue
            if end is None:
                rep.dropped.append({"orig_id": orig_id, "reason": "missing/invalid
end_time",
                                    "start": start, "end": _get(row, "end_time",
"end_mmss")})
                continue
            if end <= start:
                rep.dropped.append({"orig_id": orig_id, "reason": "end_time <= start_time",
                                    "start": start, "end": end})
                continue
            if (s_parsed and s_parsed[1]):
                rep.mmss_converted += 1
            if (e_parsed and e_parsed[1]):
                rep.mmss_converted += 1

            reason, reason_note = _primary_reason(_get(row, "ending_reason"))
            note_in = _get(row, "notes", "note", "reviewer_comment")
            notes = "; ".join([x for x in [reason_note, (str(note_in).strip() if note_in
else "")] if x])
            kept.append({
                "orig_id": orig_id, "start": float(start), "end": float(end),
                "ending_reason": reason, "sport": _get(row, "sport"),
```

```
                "shots_count": _get(row, "shots_count", "shots"), "notes": notes,
            })

        kept.sort(key=lambda r: r["start"])

        # Renumber 1..N by start order; detect whether ids were fractional / duplicate /
        # not already a clean 1..N sequence (any of which means import-local would collide).
        orig_seq = [str(r["orig_id"]) if r["orig_id"] is not None else "" for r in kept]
        new_seq = [str(i) for i in range(1, len(kept) + 1)]
        rep.renumbered = orig_seq != new_seq
        seen: Dict[str, int] = {}
        for oid in orig_seq:
            seen[oid] = seen.get(oid, 0) + 1
        rep.duplicate_ids = sorted({oid for oid, c in seen.items() if c > 1 and oid})
        rep.fractional_ids = sorted({oid for oid in orig_seq if "." in oid})

        out: List[Dict[str, Any]] = []
        for i, r in enumerate(kept, start=1):
            out.append({
                "rally_number": i,
                "start_time": f"{r['start']:.3f}",
                "end_time": f"{r['end']:.3f}",
                "ending_reason": r["ending_reason"] or "",
                "sport": r["sport"] or "",
                "shots_count": "" if r["shots_count"] is None else
str(r["shots_count"]).strip(),
                "original_rally_number": "" if r["orig_id"] is None else str(r["orig_id"]),
                "notes": r["notes"],
            })
        rep.rows_out = len(out)
        return out, rep


    def normalize_file(in_path: str, out_path: str) -> NormalizeReport:
        with open(in_path, "r", newline="", encoding="utf-8-sig") as f:
            reader = csv.DictReader(f)
            reader.fieldnames = [(n or "").lower().strip() for n in (reader.fieldnames or
[])]
            rows = [{(k or "").lower().strip(): v for k, v in r.items()} for r in reader]
        out, rep = normalize_rows(rows)
        os.makedirs(os.path.dirname(os.path.abspath(out_path)) or ".", exist_ok=True)
        with open(out_path, "w", newline="", encoding="utf-8") as f:
            w = csv.DictWriter(f, fieldnames=OUT_FIELDS)
            w.writeheader()
            w.writerows(out)
        return rep


    def main() -> None:
        ap = argparse.ArgumentParser(description="Normalize a rally-annotator CSV for safe
import-local.")
        ap.add_argument("--in", dest="in_path", required=True, help="rally-annotator
.rallies.csv")
        ap.add_argument("--out", dest="out_path", required=True, help="clean lean CSV to
write")
        args = ap.parse_args()
        rep = normalize_file(args.in_path, args.out_path)
        print("=== normalize-annotations ===")
        print(rep.summary())
        print(f"wrote -> {args.out_path}")


    if __name__ == "__main__":
        main()

```

9. user (2026-06-10T06:10:39.105Z)

```
safe_int('7.5') = 7
safe_int('7') = 7
validator passes duplicate rally_number 7/7: True []
raised: sqlite3.IntegrityError - UNIQUE constraint failed: badminton_rallies.video_id,
badminton_rallies.rally_number
pre-existing rallies after failure: [(1, 1.0)]
video row still present (committed): True
```

10. user (2026-06-10T06:10:39.508Z)

```
No matches found
```

11. user (2026-06-10T06:10:48.416Z)

```
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:52:## P0 ?
the two-landmine guard (`normalize_annotations.py`, shipped)
C:\Users\avidu\Projects\sports-data-
collector\docs\MANUAL_INGESTION_POSITIONING.md:55:`src/cli/normalize_annotations.py` pre-import
normalizer fixes both, as a **standalone
C:\Users\avidu\Projects\sports-data-collector\docs\MANUAL_INGESTION_POSITIONING.md:75:python -m
src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
C:\Users\avidu\Projects\sports-data-
collector\docs\MANUAL_INGESTION_POSITIONING.md:128:`normalize_annotations.py` step already
handles its `mm:ss` + `rally` + pipe-reason
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:4:Two verified
landmines in PARTIAL human annotation data (a video tagged only up to
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:16:     silently
destroys one rally. -> we RENUMBER 1..N by start_time, recording the
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:29:    python -m
src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
C:\Users\avidu\Projects\sports-data-collector\src\cli\normalize_annotations.py:212:
print("=== normalize-annotations ===")
C:\Users\avidu\Projects\sports-data-
collector\tests\test_normalize_annotations.py:1:"""normalize_annotations: the two-landmine guard
for partial human annotator CSVs.
C:\Users\avidu\Projects\sports-data-collector\tests\test_normalize_annotations.py:10:from
src.cli.normalize_annotations import normalize_rows, normalize_file
```

12. user (2026-06-10T06:10:49.361Z)

```
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:76:
`src/cli/normalize_annotations.py`). The normalizer guarded blank/reversed-end rows but
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:106:
`normalize_annotations.py` (the two-landmine + negative-start guard) + `docs/
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:44:Kushagra's shuttle is
visible 67.5% of the time (vs Adarsh 40%) ? tracked *through* inter-rally activity ? the
```

13. user (2026-06-10T06:10:56.926Z)

```
30          (lean 5-col CSV)          ? register (SQLite) ? export        indexer (eval/training)
31        ? manual review sheet                                           reads ONLY:
32          (rich 14-col CSV)         everything but (start,end) is        manifest.json +
33        ? vendor Roora / CVAT       validated then dropped before        per-video start,end
CSV
34          (CVAT XML / canonical)    the indexer sees it
35      ```
36
37      ## It already works today (zero code changes)
38
39      The lean annotator CSV header `rally_number,start_time,end_time,ending_reason,sport`
40      matches the `import-local` `DictReader` exactly (`curate.py::_parse_local_annotations`),
41      and the validator accepts gapped/partial coverage as-is
```

(`validator.py::validate_badminton_rallies` checks only positive duration + no overlap).
Verified end-to-end on the real file ? 40 rallies ? export ? 40-interval indexer CSV.

```bash
python -m src.cli.curate import-local --sport badminton \
  --video-path "?/Adarsh and Avi.MP4" \
  --annotations-path "?/Adarsh and Avi.rallies.csv" --license Proprietary --fps 59.94
python -m src.cli.curate export   # ? manifest.json + <video_id>.csv (start,end)
```

P0 ? the two-landmine guard (`normalize_annotations.py`, shipped)

Partial human data has two failure modes the raw path handles badly. The
`src/cli/normalize_annotations.py` pre-import normalizer fixes both, as a **standalone
step** that feeds `import-local` (it does **not** touch the DB schema or the
`rally_number`-ordered export):

1. **An unusable time row aborts the *entire* import.** A blank/missing time makes
   `safe_float_required` raise (`parsing.py`); a reversed `end_time <= start_time` or a
   negative `start_time` makes the `BadmintonRally` pydantic validator raise
   (`schemas/badminton.py::validate_timestamps`). Either way the `try/except` in
   `import-local` turns it into `sys.exit(1)`, killing the whole video. ? the normalizer
   drops such rows (mirroring the indexer loader, which already skips `not end > start`).
2. **A fractional/duplicate `rally_number` (e.g. `7.5`, `16.5`) silently corrupts data.**
   `int(float("7.5")) ? 7` (`parsing.py::safe_int`) collides on the
   `(video_id, rally_number)` primary key, and the DELETE-then-INSERT upsert
   (`db.py::insert_badminton_rallies`) destroys the colliding rally. ? the normalizer
   renumbers `1..N` by `start_time`, recording the original id for traceability.

It also converts `mm:ss`/`HH:MM:SS` ? decimal seconds and folds a pipe-delimited
`ending_reason` ("forced_error|other") to its primary token.

```bash
python -m src.cli.normalize_annotations --in "Match.rallies.csv" --out clean.csv
python -m src.cli.curate import-local --sport badminton --video-path "?/Match.MP4" \
  --annotations-path clean.csv --license Proprietary
```

Do not widen the PK to float ? the DB column is INTEGER, export orders by it, and
the indexer ignores `rally_number` entirely. Resolve ids at the adapter boundary.

Annotator ? canonical field mapping

Annotator field	Canonical	Handling
`rally_number` (lean) / `rally` (rich)	`rally_number:int`	Lean maps directly; **fractional/dup ids renumbered at the normalizer** (PK is INT)
`start_time` / `start_mmss`	`start_time:float`	Decimal direct; `mm:ss` converted to seconds
`end_time` / `end_mmss`	`end_time:float`	**Blank / ? start dropped, not raised**
`ending_reason`	`ending_reason:str?`	Vocab matches the documented canonical set (free-text, not an enforced enum); pipe-delimited folded to primary; **stored but dropped on export**
`sport`	? (from `--sport` flag)	CSV column ignored; operator sets the flag
rich-only `shots`, `true_*`, `VERIFIED`, `notes`	`shots_count` + QA/provenance	Carried where present; none reach the indexer

Roadmap to the canonical-hub end-state

All additive, video-level, backward-compatible (consumers ignore unknown manifest keys).

- **P1 ? persist the labeled window in the manifest.** Partial labels cover only a
  prefix; a full-video eval penalizes correct detections in the unlabeled tail. The

```
100       indexer already restricts to a window (`eval_partial_labels.py::restrict_to_window`)
101       but currently *infers* `window_end = max(end_time)` ? which is **wrong when a video is
102       tagged past its last rally** (valid predictions in the labeled-but-rally-free tail get
103       scored as false positives). The collector should own `labeled_window_start/end`
104       (the VLC tool is resumable + monotonic, so the session extent is known), carry it in
105       the manifest, and let the indexer apply it automatically. Short-term: pass
106       `--window-end` at eval time.
107     - **P1 ? maturity ladder + provenance.** `CurationStatus` (STAGING/CURATED/REJECTED) is
108       a *workflow* state, not a *trust* state. Add a separate `needs_review ? confirmed ?
109       gold` enum + `annotation_source` (manual_vlc / manual_review / vendor_cvat) +
110       `annotator_id` + timestamp. Route self-rated imports to `needs_review` instead of the
111       hardcoded `CURATED` (`curate.py`), so the golden-set regression flow restricts the
gold
112       tier to `maturity=gold` and an unreviewed re-tag can't silently become the answer key.
113       (Today a single-rater self-rating and a vendor-QA'd delivery are indistinguishable.)
114     - **P2 ? unify sources behind one adapter** emitting `(List[RallyRecord], List[Issue])`,
115       and run the **same Issue detectors** (gap>15s possible-missed-rally, shot-density,
116       vocab, overlap) on manual imports that vendor CVAT already gets. Today self-produced
117       labels get *less* scrutiny than a vendor delivery ? backwards.
118     - **P2 ? freeze the intervals-only export contract** in one doc both repos reference +
119       a round-trip test (header matching is exact case-insensitive, so a vendor
120       `start_seconds` would fail silently).
121
122     ## Open question
123
124     Is the **rich** `rally_review.csv` (mm:ss times, a `rally` column with fractional ids,
125     pipe-delimited reasons, `true_*` correction columns) ever the *source of truth*, or is
126     the lean VLC `.rallies.csv` always canonical? Neither `import-local` nor
127     `load_canonical_csv` (which keys on `rally_number`) ingest the rich file directly. The
128     `normalize_annotations.py` step already handles its `mm:ss` + `rally` + pipe-reason
129     quirks for the common case; a fuller converter that applies `true_*` overrides is only
130     worth building if the rich sheet is canonical.
131
```

14. user (2026-06-10T06:12:02.895Z)

```
Structured output provided successfully
```

15. assistant (2026-06-10T06:12:08.935Z)

```
Verification complete.
```